Task 1: Problem Definition and Approval

- ## Define a meaningful real-world software quality problem

Embedded computer vision (CV) systems, like those used currently in autonomous robots, drones, and smart cameras, combine resource-strained hardware (microcontrollers, single-board computers) with computationally intensive vision models (such as YOLOv8, or MediaPipe) to make real-time decisions. In lots of development workflows, quality assurance for these systems focuses almost entirely on functional requirements: does the model detect and classify objects accurately? Non-functional requirements such as latency, frame-rate stability, throughput under load, and resource usage (CPU/memory), are frequently treated as inferior concerns, tested improperly or not at all. This is an important gap. A detection that is accurate but arrives too late, or a system that degrades under load, is a genuine failure in a real time context, not just a missed opportunity.

- ## Explain the background, motivation, stakeholders, users, and practical relevance of the selected problem

- ### Background

Real time embedded CV is common in robotics, autonomous navigation, robots, and drones, where timing and resource constraints are as critical as detection accuracy. A model that performs well during the benchmark can fail in deployment if it can't keep a consistent frame rate on constrained hardware, or if performance degrades unpredictably as the system load grows. This problem is motivated by direct exposure to this context, where the disconnect between "the model works" and "the system meets its real time requirements" is a recurring, practical concern.

- ### Stakeholders and Users

Developers/Engineers – Build embedded CV pipelines, who need confidence that functionality doesn't come at the cost of performance in real time

QA/Test engineers – Need a defined, repeatable way to test NFRs alongside FRs

End users – Affected if the system misses detections due to latency or degrades under bigger loads

Safety/compliance reviewers – In contexts where late or missed detections have real consequences

- ### Practical Relevance

This is a widely common problem across all forms of robotics. Any domain where CV runs on embedded hardware under real time constraints will eventually encounter issues. Addressing it produces a testing approach that is usable well beyond a single interaction.

- ## Identify the key software quality issues involved in the problem

- ### Key Software Quality Issues

  Absence of clearly defined testable non-functional requirements for real-time CV systems

  Lack of methods to measure and validate latency, framerate, and resource usage.

  Difficulty differentiating between "the model is accurate" from "the system meets the real time requirements"

  Risk of performance degradation under load or resource allocation.

  Absence of defect management processes specifically for NFR

- ## Explain why the problem is suitable for a Software Quality Assurance Project

  The problem centres on requirements quality (defining testable NFRs), test strategy design (functional vs non-functional approaches), defect management (tracking NFR violations differently from functional bugs), and quality metrics (latency thresholds, degradation curves) which all are core SQA requirements. It also supports a requirements traceability matrix, since both functional and non-functional requirements can be trace to measurable test cases.

- ## Confirm that the project is achievable within one semester

  The project is a software-simulated prototype rather than physical embedded hardware, removing cost and hardware-access constraints. By using pre-recorded video/image datasets, an already existing lightweight CV mode, and simulated resource constraints, we can achieve the technical build within a few weeks, leaving the rest of the semester for QA artifacts (requirements analysis, test strategy, test cases, traceability matrix, defect management and metrics) which is where the assessment is mostly centred around.